Solving Polynomial Systems of using Linear Elimination

Ilay Menahem, Michael Shalyt, Ido Kaminer

March 22, 2026

Abstract

There is a problem shared among many fields of science of solving polynomial systems of equations. The required solution is often a space of solutions, rather than a single solution. Applications include QFT, robotics, cryptography, optimization, chemistry, electrical engineering, and more. [we need other examples... best is to have concrete examples from history. If not then maybe say generally that in many areas of science and engineering a parametric solution can surprise us and help where our creativity is limited].

This paper presents a scalable pipeline for finding a space of solutions (of full dimension) to polynomial systems of equations. We start from numerical solutions, transitioning to symbolic solutions, and finally parametrize the solution space via structured exploration. We then also discuss other complementary methods and discuss what open questions still remain. We demonstrate these methods on concrete examples from the field of QFT.

Introduction

Solving polynomial systems of equations is a fundamental problem in mathematics and computer science, with applications in fields such as robotics, cryptography, optimization, QFT, chemistry, electrical engineering, economics and more. In QFT, polynomial systems arise in the context of generilizing and unifying theories (e.g. quantum gravity). In chemistry, for example, polynomial systems appear in the study and design of reaction chains. In robotics, for example, polynomial systems are used to model kinematics and motion planning.

In most of these use cases, the required solution is often a space of solutions, rather than a single solution. Specifically, in QFT the proposed theory must follow

gauge invariance, which leads to a space of solutions, and of these solutions one may want to find the one represents the simplest theory. In chemistry, reaction chains often have multiple valid pathways, leading to a solution space. In robotics, the solution space may represent all possible configurations of a robotic arm that satisfy certain constraints. one would want to work with the entire solution space to find optimal paths and configurations. This type of problem is universal across many fields of science and engineering. It calls for a comprehensive approach that can efficiently find and represent the solution space of polynomial systems.

Classic methods for solving polynomial systems of equations include Groebner bases [], resultants [], and homotopy continuation []. However, these methods can be computationally expensive and may not scale well with the size of the system. Other approaches, such as gradient [], and SMT solvers [] are focused on finding specific solutions rather than exploring the entire solution space.

In this paper, classify/group/review/discuss specially made approaches for finding the general ... in efficient ways relevant for. We also demonstrate the methods on concrete examples from QFT. Among those methods, we propose a pipeline that leverages numerical methods to find initial solutions, then refines these solutions into symbolic forms, and finally explores a space of solutions. This approach aims to combine the efficiency of numerical methods, and some insights we may have at how the symbolic solutions may look like, to provide a comprehensive understanding of a solution space of polynomial systems. We also present and analyze a complementary examples from QFT, showcasing the effectiveness of our approach in practical scenarios. We finally outline the open questions and future directions [more concrete] in this field.

Mathematical Background

Let $\mathbb{K}[\mathbf{x}]$ denote the polynomial ring in variables $\mathbf{x} = (x_1, \ldots, x_n)$ over a field $\mathbb{K}$, typically $\mathbb{F}_q$, $\mathbb{R}$ or $\mathbb{C}$. A polynomial system is defined by a map $F : \mathbb{K}^n \to \mathbb{K}^m$ with components $f_1, \ldots, f_m \in \mathbb{K}[\mathbf{x}]$. The locus of simultaneous solutions constitutes the *affine variety* $V(F) = \{\mathbf{a} \in \mathbb{K}^n \mid F(\mathbf{a}) = \mathbf{0}\}$.

In this paper, we will mostly work over the rational, real or complex numbers. given a continuously differentiable system of equations $F \in C_1$ (possibly polynomial) with a solution point p, the dimension of the solution space in the neighborhood of p is $n - rank(J_F(p))$ where J is the Jacobian matrix of F at point p [1].

Linear elimination

even in the case of very large non linear polynomial systems of equations, it is possible to eliminate variables in a linear fashion, which can significantly reduce the size of the system and the number of variables.

Algorithm 1 Linear elimination by substitution

Require: Polynomial system $P = \{p_1, \ldots, p_k\} \subseteq \mathbb{K}[x_1, \ldots, x_m]$
Ensure: Reduced system Q where no $q \in Q$ is linear in any remaining variable; and a substitution map Σ to recover eliminated variables

$Q \leftarrow P$
$V \leftarrow \{x_1, \ldots, x_m\}$
$\Sigma \leftarrow \emptyset$
$L \leftarrow \{(x', p') \mid x' \in V,\ p' \in Q,\ p' \text{ is linear in } x'\}$
while $L \neq \emptyset$ **do**
    Select any pair $(x, p) \in L$
    Solve $p = 0$ for x to obtain $x \leftarrow f(\hat{x})$
    $\Sigma \leftarrow \Sigma \cup \{x \mapsto f(\hat{x})\}$
    Substitute $x \leftarrow f(\hat{x})$ in Q
    $V \leftarrow V \setminus \{x\}$
    clear Q of zero equations
    $L \leftarrow \{(x', p') \mid x' \in V,\ p' \in Q,\ p' \text{ is linear in } x'\}$
end while
return (Q, Σ)

we have the freedom to choose which linear variable to eliminate at each step. good heuristics to choose which variable to eliminate include:

- **Lowest degree in the other equations:** eliminate the variable that appears with the lowest degree in the other equations first.
- **Lowest degree after substitution:** eliminate the variable that will lead to the lowest degree in the other equations after substitution.
- **Greatest or least number of occurrences:** eliminate the variable that appears in the greatest or least number of equations first.

It is also possible to use DFS or BFS to search on the order of elimination.

System dataset

For linear elimination we need to be able to do the following operations efficiently

- is polynomial linear in a variable
- is polynomial zero
- linearly eliminate a variable from a polynomial

To prevent an exponential blowup in the size of the equations, we store the current polynomial system in unexpanded form. Each polynomial expression is represented as a DAG, so common subexpressions are shared and elimination steps do not force an immediate expansion of all dependent equations. The algorithm therefore works directly with graph-represented polynomials.

Once the system is stored in this form, linearity and vanishing can no longer be decided from the top-level syntax alone. Instead, the required operations are phrased in terms of evaluation of the polynomial represented by the DAG.

For the operation "is polynomial zero", we evaluate the represented expression at sampled points of its remaining free variables. If every sampled value is zero, we treat the equation as zero and discard it from the active system.

Lemma 1 *If we sample n points from a polynomial $p(x)$ and all the sampled values are zero, then $p(x) \equiv 0$ with probability 1.*

Proof 1 — *To be added* —

For the operation "is polynomial linear in a variable", let p be one of the remaining equations and let x be a variable that has not yet been eliminated. The represented polynomial is viewed as a function of x and of the other still-free variables. To determine whether it is linear in x, we fix the other variables and evaluate the expression at several values of x. If the sampled values lie on a single affine line, then the represented expression behaves as $ax + b$ in the variable x, and x is a valid candidate for linear elimination.

Lemma 2 *If we sample n points from a polynomial $p(x)$ and the sampled values lie on a single affine line, then $p(x) = ax + b$ for some constants a and b with probability 1.*

Proof 2 — *To be added* —

For the operation "linearly eliminate a variable from a polynomial", suppose that the represented equation is linear in x, so it can be written as

$$ax + b = 0,$$

where x is the variable to be eliminated. Since the equation is linear in x, both a and b may depend on the other variables but not on x. These coefficients can be recovered by evaluation: $b = p(0)$ and $a = p(1) - p(0)$. If $a = 0$ and $b \neq 0$, then the attempted elimination step is illegitimate, because the equation cannot be solved for x, if $b = 0$ we discard the equation. Otherwise, the elimination step solves for x and records the resulting expression as part of the unexpanded system:

$$x \mapsto -\frac{b}{a} = \frac{p(0)}{p(0) - p(1)}.$$

The original equation is then removed from the remaining system, while the solved form is retained as a DAG-represented expression rather than applied eagerly to every other equation.

Evaluating a DAG-represented expression

Let G be a DAG representing a polynomial expression, and let α be an assignment of values to the variables occurring in G. To evaluate the represented expression, we process the nodes of G in topological order, so that every child of a node is evaluated before the node itself.

Algorithm 2 Evaluating a polynomial expression represented as a DAG

Require: DAG G with root r, and an assignment α of values to the variables of G
Ensure: The value of the polynomial represented by G at α

for each node v of G in topological order **do**
  if v is a constant node with value c **then**
    $\mathrm{val}(v) \leftarrow c$
  else if v is a variable node labeled by x **then**
    $\mathrm{val}(v) \leftarrow \alpha(x)$
  else if v is an addition node with children $u_1, \ldots, u_k$ **then**
    $\mathrm{val}(v) \leftarrow \mathrm{val}(u_1) + \cdots + \mathrm{val}(u_k)$
  else if v is a multiplication node with children $u_1, \ldots, u_k$ **then**
    $\mathrm{val}(v) \leftarrow \mathrm{val}(u_1) \cdots \mathrm{val}(u_k)$
  else if v is a negation node with child u **then**
    $\mathrm{val}(v) \leftarrow -\mathrm{val}(u)$
  end if
end for
return $\mathrm{val}(r)$

Because the expression is represented as a DAG rather than a tree, a shared subexpression appears as a single node and is evaluated only once. The values computed at intermediate nodes can therefore be reused wherever that subexpression occurs, which makes repeated evaluations efficient and supports the tests for linearity and vanishing described above.

Handling systems without linearities

the polynomial order we will use to interreduce, and make polynomials monic, should maximize the number of monomials with that are linear in one of any of the variables. thus we should use the grevlex order.

References

[1] Stepan Yu Gatilov. Using low-rank approximation of the jacobian matrix in the newton–raphson method to solve certain singular equations. *Journal of Computational and Applied Mathematics*, 272:8–24, 2014.

Appendix

Simplifying expressions

To simplify an expression, we can use the following local simplifications:

- $x + 0 \mapsto x$
- $x - x \mapsto 0$
- $-(-x) \mapsto x$
- $1 \cdot x \mapsto x$
- $0 \cdot x \mapsto 0$
- $\frac{x}{x} \mapsto 1$
- $\frac{x}{1} \mapsto x$
- constant propagation